RESCU- Rapid Emergency Sensor Communications Unit

Reed Baker, Alexis Haas, Ishana Thakre, Haejune Kwon, Jacob Hilton

22 February 2026

GTA: Yuhang Zhu

Professor: Ryan Beasley, Dr. Carla Zoltowski

Design Document

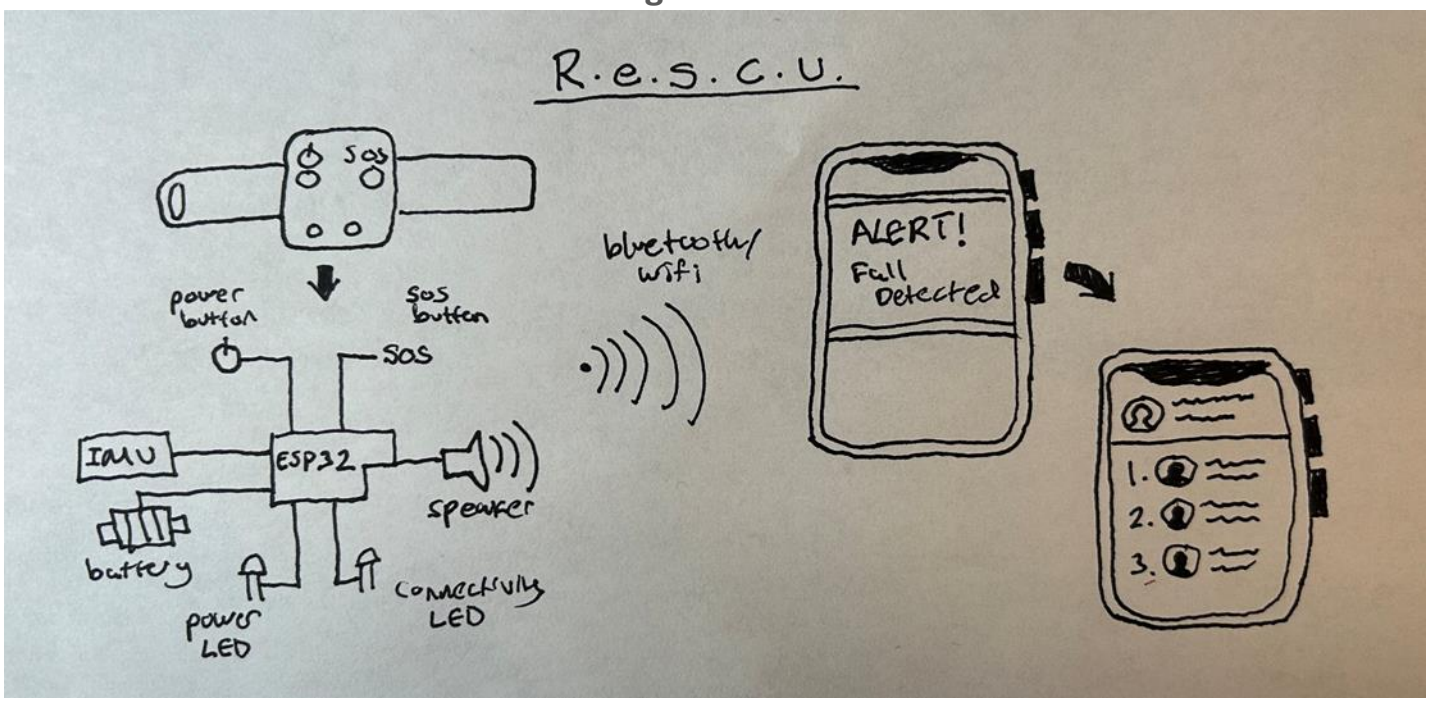

Figure 1. Product Sketch

Contents

Introduction 6
Executive Description 6
User Story 6
Design Specifications 7
Specifications 7
Factors influencing specifications 7
Public Health, Safety, and Welfare 7
Global Factors 8
Social Factors 8
Environmental Factors 8
Economic Factors 8
System Overview 9
System Block Diagrams 9
System Activity Diagram 9
System Mechanical Design (Extra Credit) 10
Integration Approach 10
System Photographs 10
Subsystems 10
Subsystem 1: Hardware design – Jacob Hilton 10
Subsystem Diagram 10
Specifications 10
Subsystem Interactions 12
Core ECE Design Tasks 12
Schematics 12
Parts 13
Algorithm 10
Theory of Operation 14
Specification Measurements 14
Standards 14
Subsystem 2: Microprocessor Interface – Haejune Kwon 15
Subsystem Diagram 15
Specifications 15
Subsystem Interactions 16
Core ECE Design Tasks 16

Schematics 16
Parts 17
Algorithm 17
Theory of Operation 18
Specification Measurements 18
Standards 18
Subsystem 3: Application Front-End – Alexis Haas 24
Subsystem Diagram 24
Specifications 24
Subsystem Interactions 25
Core ECE Design Tasks 25
Schematics 25
Parts 26
Algorithm 12
Theory of Operation 26
Specification Measurements 26
Standards 26
Subsystem 4: Application Back-End – Ishana Thakre 27
Subsystem Diagram 27
Specifications 27
Subsystem Interactions 28
Core ECE Design Tasks 28
Schematics 28
Parts 29
Algorithm 29
Theory of Operation 29
Specification Measurements 30
Standards 30
Subsystem 5: Application Network Communication – Reed Baker 14
Subsystem Diagram 14
Specifications 14
Subsystem Interactions 14
Core ECE Design Tasks 14
Schematics 14
Parts 14

Algorithm 14
Theory of Operation 14
Specification Measurements 14
Standards 14
PCB Design 30
PCB Schematics 30
PCB Layout 30
Final Status 30
Team Structure 30
Team Member 1 30
Team Member 2 31
Team Member 3 31
Team Member 4 31
Team Member 5 31
Bibliography 31
Appendices 35
Appendix A: [NAME] 35

Revision Log

Date	Revision	Changes
02/08/2025	v0.1	• Added Introduction • Added Design Requirements • Added System Block Diagram • Added System Activity Diagram • Added Subsystem Parts • Added Subsystem Standards • Added Team Structure • Added Bibliography
02/22/2025	v0.2	• Updated System Block Diagram • Added Design Sketch 2.0 • Added Subsystem Block/Activity Diagram • Added Subsystem Specifications • Added Subsystem Interactions • Added Subsystem Core ECE Design Tasks • Added Subsystem Schematics/Code • Updated Subsystem Parts • Added Subsystem Algorithm • Added Subsystem Theory Of Operation

Introduction

Executive Description

Family members often worry about the safety of older and elderly family members who live alone. One of the most significant risks faced by the elderly population is fall-induced injury, as falls are the leading cause of both fatal and nonfatal injuries among older adults. [1] Mobility-related challenges become worse with age, which leads to this increased risk of falling for the elderly population. As a result, families want a proper and reliable safety net in place to protect their elderly family members that live alone.

Currently, medical alert devices are the most common solution for this safety net. These devices detect when a user has fallen and notify the proper emergency medical services. However, the options available currently on the market are extremely inconsistent, expensive, and confusing to set up.

To address these problems, we are proposing an app-paired safety device that consistently monitors the fall status of our users and then contacts the user's prechosen emergency contacts upon a fall event. Our device will offer a wider range of customization options, ease of setup, and affordability to make sure that our users and their families can have the peace of mind that they desire with their safety net. In this customization, the user will be able to input a list of emergency contacts that will be contacted in priority order that the user chooses. Additionally, since our device will pair with the user's mobile phone, the device will be able to be used both indoors and outdoors, which is an advantage over other market-available options.

As for the aspects and components of our system, we are proposing a wearable arm band that contacts a microcontroller, an inertial measurement unit (IMU), and a barometer. The IMU and barometer will both be used for fall detection, to determine when rapid acceleration, orientation changes, and impact events associated with falls. The microcontroller will process this data from the sensors, determine if a fall has occurred, and then notify the mobile application if a fall event has occurred. The phone application will contact pre-selected emergency contacts when a fall event has occurred, and the user is in danger.

User Story

As a grandson, Reed is concerned about his grandmother's safety. With age, she has experienced increasing mobility issues. Mobility issues are defined as any trouble walking around and/or going up and down stairs. According to the CDC, around 12.2% of all US adults (around 31.5 million people) have mobility issues of some kind [1].

Reed and his family decided to investigate getting her some kind of fall detection device. They were worried about her potentially falling while she was home alone and not having anyone to help her up or call emergency services if she was injured. However, when they started looking into options, they quickly realized that none of the available options came without significant drawbacks.

The first of the two main options available to them were Life Alert [2] and similar services. The main issue that they ran into with these programs was that the programs were expensive and not very accessible. Life Alert doesn't offer any actual pricing on their website; they only offer a quote service where personal information is needed and then they get back to you with a quote. Life Alert also comes with an installation fee of over one hundred dollars [3]. On top of this, there is a monthly service fee to use the device.

The other option that they tried was mobile apps. There are several apps that advertise themselves as fall detection apps on the Appstore. Reed and his brother downloaded apps like FallCall Detect [4] and SeizAlarm [5] and tried to get them to trigger. They threw themselves onto couches and mattresses and couldn't get the apps to trigger a single time. The largest problem with the apps was that they couldn't get the fall detection to trigger at all.

Frustrated by existing options, Reed's family needs a fall-detection device that works. They are looking for a high-accuracy, low-maintenance solution that fits a modest budget. By prioritizing accuracy, ease of use, and affordability, they hope to ensure her safety and independence and have peace of mind about her health.

Design Specifications

Requirements

- The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positives.
- The device shall be compact and comfortable enough to wear for full days of use, without impeding day-to-day life.
- Upon detection of a potential fall event, the system shall display an alert on the mobile application and wait for a configurable delay period during which the alert may be canceled.
- The system shall allow the user to manually initiate an emergency alert independent of automatic fall detection.
- The system shall provide audible and visual feedback indicating device power status, connectivity status, battery condition, and confirmed emergency events.
- The system shall maintain reliable wireless communication between the wearable device and the paired mobile application within an expected residential operating range.
- Upon confirmation of a fall event, the system shall securely transmit a complete and actionable alert payload to backend to initiate emergency response procedures.
- The system shall notify the configured emergency contacts in a defined priority order until an affirmative response is received.
- The system shall log fall events, device status data, and emergency communication outcomes for later retrieval by authorized users.
- The system shall protect and manage stored user and device data using secure, authenticated, and scalable infrastructure.
- The system shall allow users to create, modify, and delete emergency contact information and configure alert preferences.
- The system shall have a robust and well-made mobile application that is user-friendly.
- The system shall continuously monitor device status, including battery level and connectivity.
- The system shall support a minimum of one full day of typical operation between required recharging cycles.

Factors influencing specifications

Public Health, Safety, and Welfare

1. **Accuracy and automation:** The device must accurately detect falls in elderly users and automatically message emergency contacts for situations associated with immobility.
2. **Minimize false detection:** The system must minimize false positives and false negatives to ensure user trust and avoid unnecessary emergency responses that could cause social distress or resource misuse.

Global Factors

1. **Wireless compatibility:** The design must be adaptable to different regions by supporting common wireless standards (Bluetooth and Wi-Fi) and compatibility with widely used smartphones.
2. **Scalable accessibility:** The system should be low-cost and scalable to increase accessibility for elderly populations worldwide, including in regions with limited access to expensive medical alert systems.

Social Factors

1. **User comfort hardware:** The wearable device must be lightweight, comfortable, and discreet to encourage consistent daily use without stigmatizing the user.
2. **User-friendly application:** The mobile application should be simple and intuitive, so elderly users and caregivers can easily configure emergency contacts and monitor device status.

Environmental Factors

1. **Energy efficient hardware:** The device must be energy-efficient to support daily use with a rechargeable Li-Po battery and reduce electronic waste through a long battery lifespan.
2. **Durable hardware:** The enclosure should be durable and resistant to sweat, minor impacts, and daily environmental exposure to ensure reliable operation in real-world conditions.

Economic Factors

1. **Low manufacturing cost:** The use of an IMU and ESP32 Feather Pico allows the system to meet functional requirements while keeping component and manufacturing costs low.
2. **Rechargeable battery:** Nightly recharging with a Li-Po battery eliminates recurring costs associated with disposable batteries, making the system more affordable over time.

System Overview

System Block Diagrams

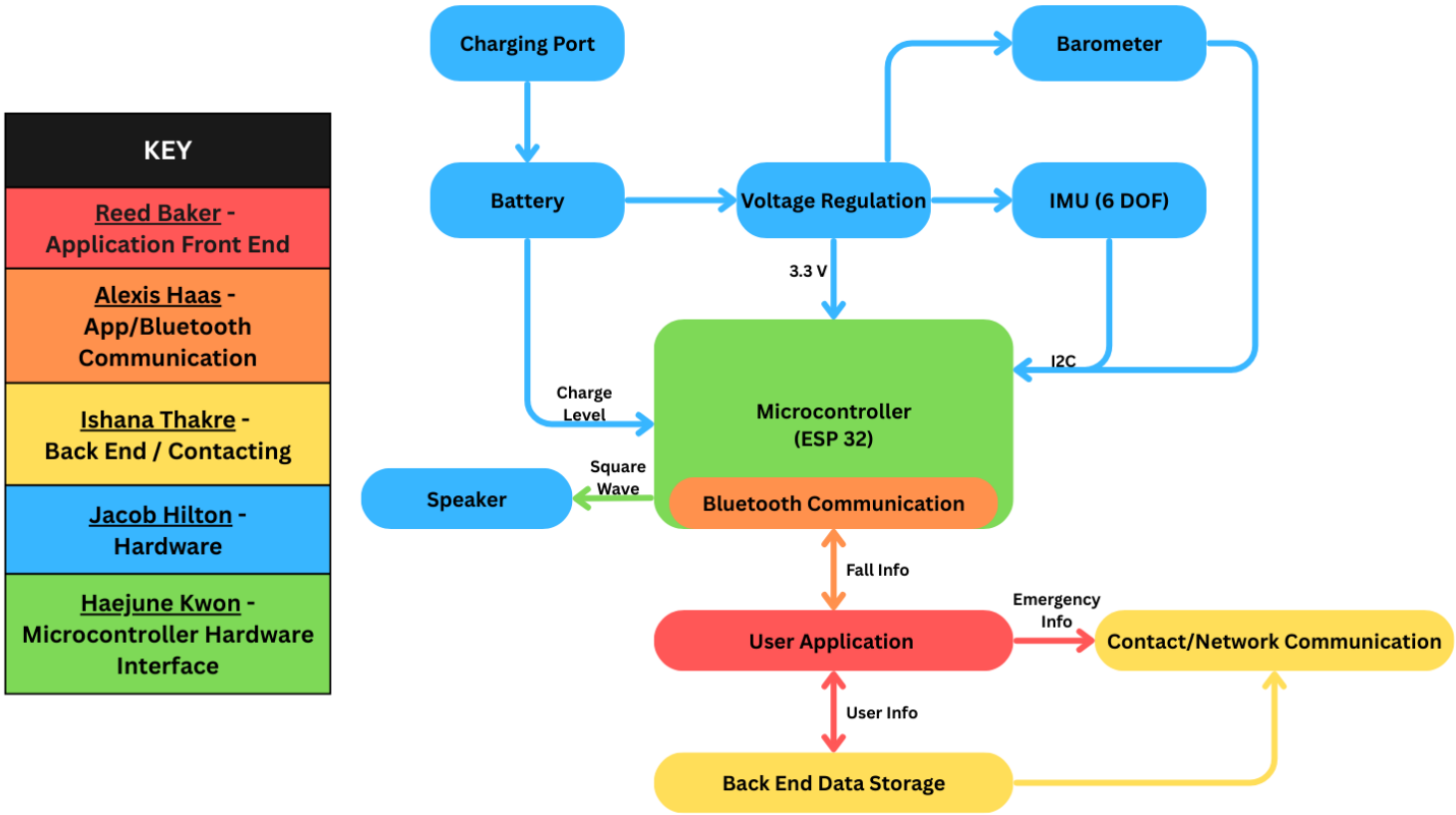

Figure 2. System Block Diagram (with Subsystem Ownership)

System Activity Diagram

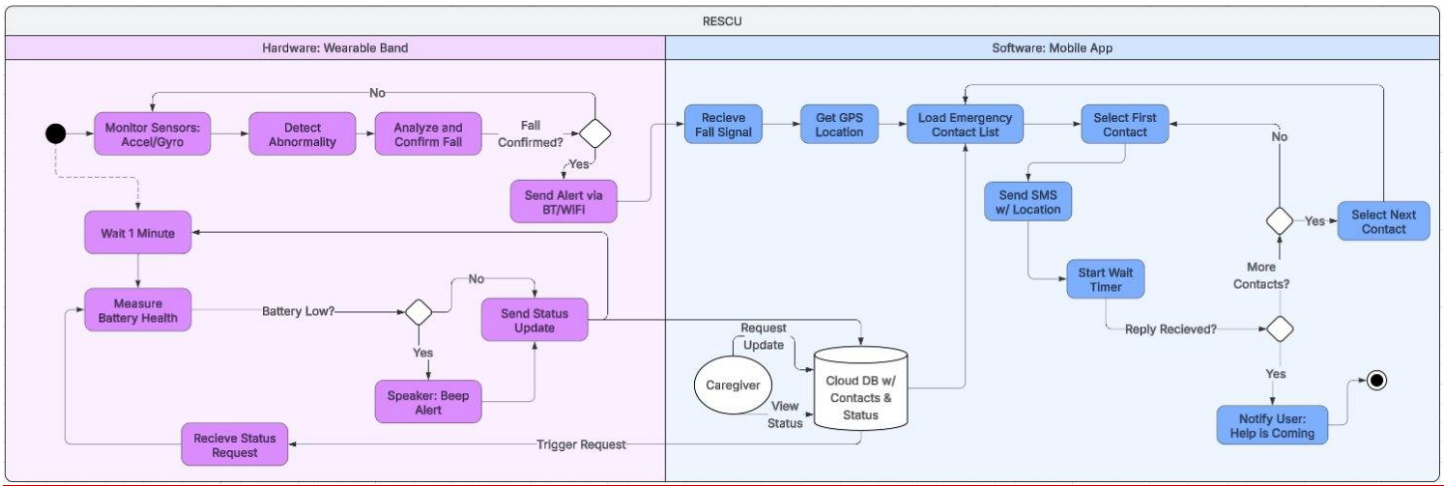

Figure 3. System Activity Diagram

System Mechanical Design (Extra Credit)

[Design Document 3]

Integration Approach

[Design Document 3] [Theory behind the system design, with reference to subsystem integration within your system]

System Photographs

[Design Document 3]

Subsystems

Subsystem 1: Hardware design – Jacob Hilton

Subsystem Diagram

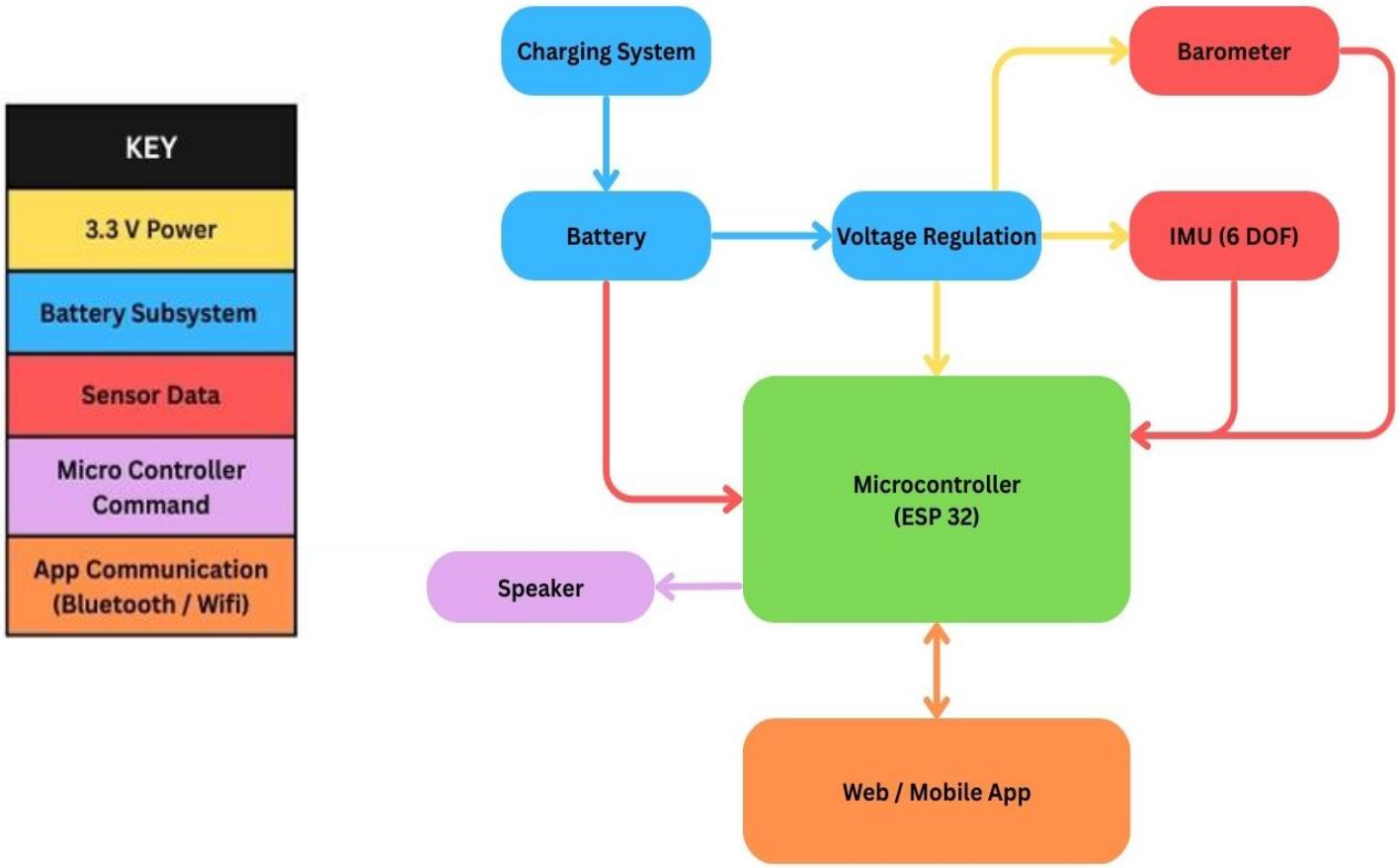

Figure 4. Hardware Block Diagram

Specifications

- **Specification 1:** The device will detect falls with at least 85% accuracy, with a less than 10% false negative and false positive rating.

    - **Requirement:** The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positives.
    - **Justification:** User safety is the primary purpose of this project. As such, accuracy of fall detection is paramount to the functioning and marketing of our project. Our users expect and depend on this device to provide the safety and accuracy that is advertised, it is integral that this specification is respected. Regarding false positives, similarly, we want our device to function as expected. Users will not trust us or even ignore the device if there are frequent false positive falls detected.
- **Specification 2:** The device will have at least an average of 3.75 / 5 on a Likert Scale survey over the device's comfort, to be conducted by sample patients.
    - **Requirement:** The device shall be compact and comfortable enough to wear for full days of use, without impeding day-to-day life.
    - **Justification:** Since this is a medical safety device, it is very important that the device is not intrusive. If the device was too intrusive and impeded day-to-day life, our users would not be motivated to wear and use the device. As such, it is important that we take considerable effort to make the device as small and comfortable as possible. Surveying actual users of our product is the best way to measure the comfort of the device, by looking at metrics (scored from 1-5) of physical comfort, skin irritation, restriction of movement, weight, and noticeability throughout the day.
- **Specification 3:** The device's battery should last for at least 15 hours of regular use.
    - **Requirement:** The system shall support a minimum of one full day of typical operation between required recharging cycles.
    - **Justification:** The battery will be powering the whole system, so it is important in that it is needed for the entire device to function properly. Since our users expect accurate fall detection, we want the battery to be able to last all day, without random battery death or dropout. Regarding rechargeability, it is not feasible to depend on the user to monitor battery life and change out the batteries when needed. It makes much more sense to just have the user plug in the device every night and allow it to charge while they sleep.
- **Specification 4:** The device's battery should be able to reach full charge within 6 hours of charging.
    - **Requirement:** The system shall support a minimum of one full day of typical operation between required recharging cycles.
    - **Justification:** We want our device to be chargeable by being plugged in, as we expect our users to just plug the device into a wall outlet when they go to bed at night. Capping the charge time at 6 hours makes it so that even for our users that only sleep 6 hours, they can still have a fully charged device available for them in the morning.
- **Specification 5:** The IMU and barometer will be characterized and calibrated to determine different types of falls, with an overall sensor accuracy of at least 90%, and an accelerometer range of ±8 g.
    - **Requirement:** The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positives.
    - We cannot depend on the user to calibrate the device every time that they want to use it, so our sensors need to be able to take accurate measurements of the surroundings without requiring any calibration past the initial calibration. Additionally, drift is a major concern, specifically for the IMU, as with continued use, the gyroscope has increasing amount of drift. We will need to find a way to combat this to continue to provide accurate fall detection. Also, the ability to characterize falls based on their impact/potential for danger will be beneficial to improving the user's safety and experience.
- **Specification 6:** The speaker will make a loud, audible alert of at least 85 dB SPL when measured 1 meter away.

- **Requirement:** The system shall provide audible and visual feedback indicating device power status, connectivity status, battery condition, and confirmed emergency events.
- **Justification:** The speaker is used to help the user's defined emergency contacts locate them, to provide the assistance needed to keep the user safe. As such, it is important that the speaker is loud enough to be heard from at least a couple rooms away so that the helper can find and locate the user. Additionally, we want to make sure that the user is away that the alert system has been triggered, in the case of a false alarm, so that it may be cancelled.

Subsystem Interactions

This subsystem is the core of the entire device. It will power all the sensors and the microcontroller that is used by the Microcontroller Hardware Interface Subsystem, which will be used to detect falls. This fall data is then what is used by all the application subsystems. Additionally, all external hardware is part of this, including the housing and the wearable band.

Core ECE Design Tasks

- ME 27000 – Mechanics of falls for sensor characterization
- ECE 20001 / 20002 – Circuit design
- ECE 20007 / 20008 – Power design and buck converter design
- ECE 36200 – Microcontroller, GPIO use, I^2C communication
- ECE 44000 – Sensor data transmission

Schematics

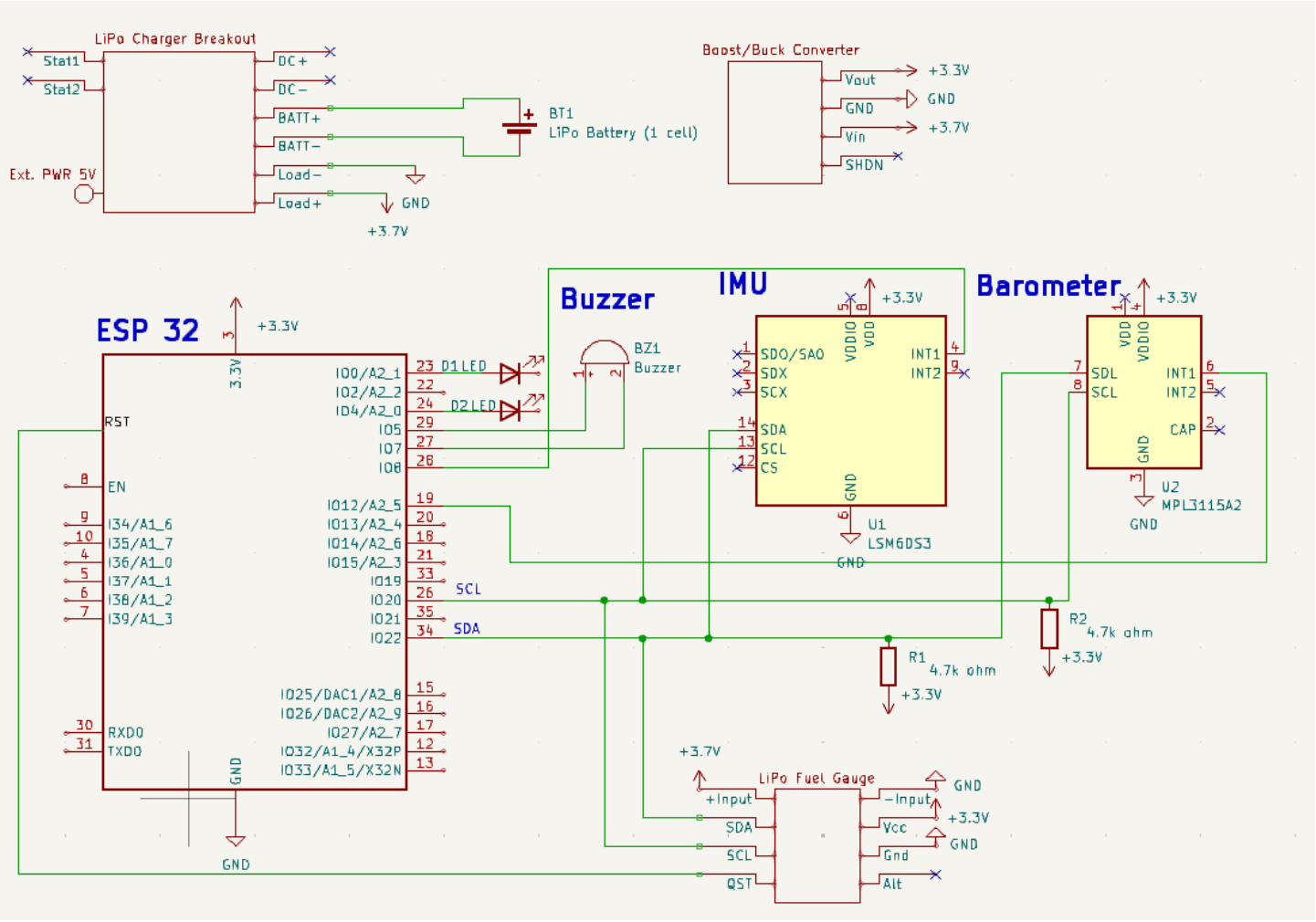

Figure 5. Hardware Circuit Diagram

Parts

- ESP32 Feather Pico microcontroller [1]
- 6-DOF IMU sensor [2]
- 1 Barometer [3]
- 2 RGB LEDs
- 2 pushbuttons
- 1 Piezo buzzer – PS1240 [4]
- LiPo battery [5]
- Charging port [6]
- LiPo Fuel Gauge [7]
- Buck/Boost Converter [8]

Theory of Operation

At the core of this subsystem is the power management components. For this device, we chose a 3.7v, 2500 mAh lithium-ion battery as its power source. This battery was chosen because of its small size, as well its battery life. Using generous estimates for power draw, we believe that this battery should last for at least the specified 15 hours of use, even with rigorous use. This battery will be connected to a "LiPo fuel gauge" from SparkFun that will communicate with our microcontroller over I^2C to show battery life/health. It also sends a signal once the battery has dropped below a certain percentage of health. We will be able to use these parameters to notify the user when their device is reaching an unsafe level of charge, so that they know that the device needs to be charged. The other integral part of the power management assembly is the buck/boost converter. Since the battery is outputting an unsteady 3.7v, while the microcontroller and sensors are expecting 3.3v, the buck/boost converter will be able to convert the unsteady signal to a constant 3.3v. This will ease electrical wear-and-tear of the components and remove variance from the overall system.

To charge the system, we are using a regulated USB mini-B charging board (MCP73833). This charging board has a variable charging rate that we will set to 500-mA, which should allow the battery to be fully charged within 5 hours. The 500-mA charging rate will keep the battery healthy, as well as the other components healthy (as they will be disconnected during charge operation). Additionally, this charging board has overvoltage and overheating protection to keep the user safe during the charging process.

As for the detection of user fall events, a 6-DOF inertial measurement unit (IMU) and a barometer will collect data to be processed by the ESP32 Feather Pico microcontroller. The focus of the microcontroller is discussed in the Microprocess Hardware Interface Theory of Operation. The priority of this subsystem's interactions with the sensors is to calibrate and characterize these sensors to provide the most accurate detection possible.

For the IMU, three degrees of linear acceleration and three degrees of angular velocity are always recorded. This will be our primary metric to categorize the intensity of a fall event. Extensive characterization of this sensor and its sensitivity will be needed to have an accurate detection method. A rapid change in any of the three dimensions will usually indicate that the user has fallen. The IMU is able to measure these accelerations by sensing the inertial forces that are acting on it and translating it into data that can be processed by the microcontroller, via I^2C communication protocol. We will be able to categorize the intensity of these falls (thus their danger levels) by characterized rapid changes in acceleration and angular velocity. Power for this sensor will be supplied by the buck/boost converter, at 3.3v.

Our additional source of fall data will come from the barometer. The barometric sensor measures the air pressure surrounding it. If a fall event occurs, the elevation of this sensor would rapidly change, causing the air pressure being measured to rapidly change too. When a rapid change in air pressure has happened, that will also be measured as a sign that a fall event may have happened. Similar to the IMU, characterization will be performed to have the best accuracy and make sure both datasets in conjunction can categorize and determine fall events with a high accuracy. Also, like the IMU, this sensor receives its power from the buck/boost converter, at 3.3v. It will also use I^2C communication protocol to communicate with the microcontroller for data processing.

Specification Measurements

[Design Document 3][Every specification above should be validated with measurements]

Standards

- **IEC 62368-1:** Provides safety guidelines for battery-powered wearable electronic devices, addressing electrical, thermal, and mechanical hazards to protect users. [7]
- **I^2C communication:** Defines the communication protocol used between the IMU sensor and the ESP32 microcontroller, enabling low-power and reliable sensor data transfers through clock and data signals. [8]

[Design Document 3][Finalized]

Subsystem 2: Microprocessor Hardware Interface – Haejune Kwon

Subsystem Diagram

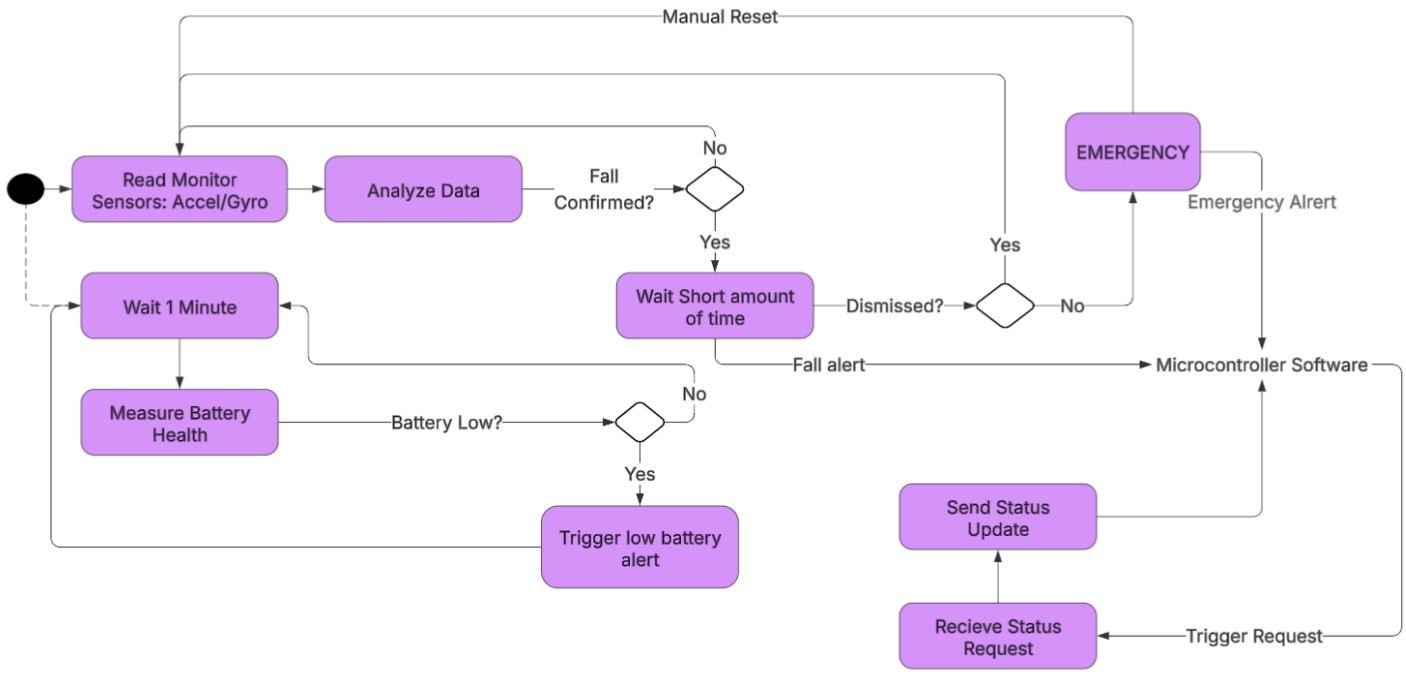

Figure 6. Microcontroller Hardware Interface Activity Diagram

Specifications

- **Specification 1:** The system shall support a minimum sensor sampling rate of 50 Hz with an I^2C data rate of at least 20 kbps.
    - **Requirement**: The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positive; The system shall continuously monitor device status, including battery level and connectivity.
    - **Justification:** Typical human falls occur within approximately 0.5–0.7 seconds [1]. A 50 Hz sampling rate provides 25–35 samples during the duration of a fall event, which is sufficient to capture rapid acceleration spikes and orientation transitions necessary for reliable fall detection. The 6-DoF IMU and the barometer requires total of18 bytes per sampling instance. This corresponds to 900 bytes per second of payload data. Accounting for I^2C protocol overhead and the data from the fuel gauge, a minimum data rate of 20 kbps ensures reliable real-time data acquisition without missed samples or bus saturation.
- **Specification 2:** The fall detection algorithm shall process sensor data using a 1.5-second sliding window for classification.
    - **Requirement**: The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positives.
    - **Justification**: Although a typical fall occurs within approximately 0.5–0.7 seconds from loss of balance to ground impact, accurate classification requires capturing the full motion sequence, including the pre-fall phase, impact event, and post-impact inactivity period. A 1.5-second processing window ensures that the complete fall dynamics are contained within a single evaluation interval rather than isolating only the impact spike, which may resemble abrupt but non-

harmful daily movements. By using a window significantly longer than the expected fall duration, the algorithm can evaluate temporal context, motion cessation, and stabilization behavior, thereby improving detection reliability and reducing false positives.

- **Specification 3:** The device shall include a minimum of 6 GPIOs, with PWM capability available on output pins.
    - **Requirement**: The system shall allow the user to manually initiate an emergency alert independent of automatic fall detection; The system shall provide audible and visual feedback indicating device power status, connectivity status, battery condition, and confirmed emergency events.
    - **Justification**: The system requires two dedicated GPIO inputs for push buttons to enable manual emergency initiation and user cancellation of alerts. Two GPIO outputs are required to drive status LEDs for visual feedback. One GPIO is required to drive the buzzer for audible alerts, and an additional GPIO is allocated to support independent control of the buzzer or expanded signaling functionality. PWM capability is necessary to generate square-wave signals for the buzzer and to modulate LED brightness for differentiated visual indications. Providing separate GPIOs with PWM support ensures reliable and independent control of user input and feedback components.
- **Specification 4:** The device shall generate two distinct alert signals separated by a configurable delay of 2–10 seconds.
    - **Requirement**: Upon detection of a potential fall event, the system shall provide the user with a configurable delay period during which the alert may be canceled.
    - **Justification:** A two-stage alert mechanism reduces false positives while preserving rapid emergency response capability. Upon initial fall detection, the first alert signal notifies the connected application to prepare emergency handling procedures. During the subsequent configurable delay period of 2–10 seconds, the user may cancel the alert through manual input. If the alert is not dismissed within this interval, a second confirmation signal is issued, indicating that the fall event is likely a true emergency. This staged signaling approach enables user intervention, minimizes unintended emergency escalation, and ensures that confirmed alerts are transmitted only after verification through timed user response.

Subsystem Interactions

This subsystem interfaces with the **hardware** and **network communication** subsystems. The microcontroller receives a regulated 3.3 V supply from the hardware subsystem, and all sensors, user inputs, and output components are physically connected through the hardware design. The microcontroller acquires sensor data, executes the fall detection algorithm, and generates the appropriate alert signals upon detection of a fall event. These alert signals are transmitted to the network communication subsystem, which manages the emergency protocol within the mobile application. Additionally, when status information is requested by the network communication subsystem, the microcontroller provides the relevant device data, including sensor readings and system status information.

Core ECE Design Tasks

- ECE 36200 – I^2C communication
- ECE 40862 – Timer for microcontroller
- ECE 40862 – I/O control
- ECE 40862 – sensor data processing

Schematics

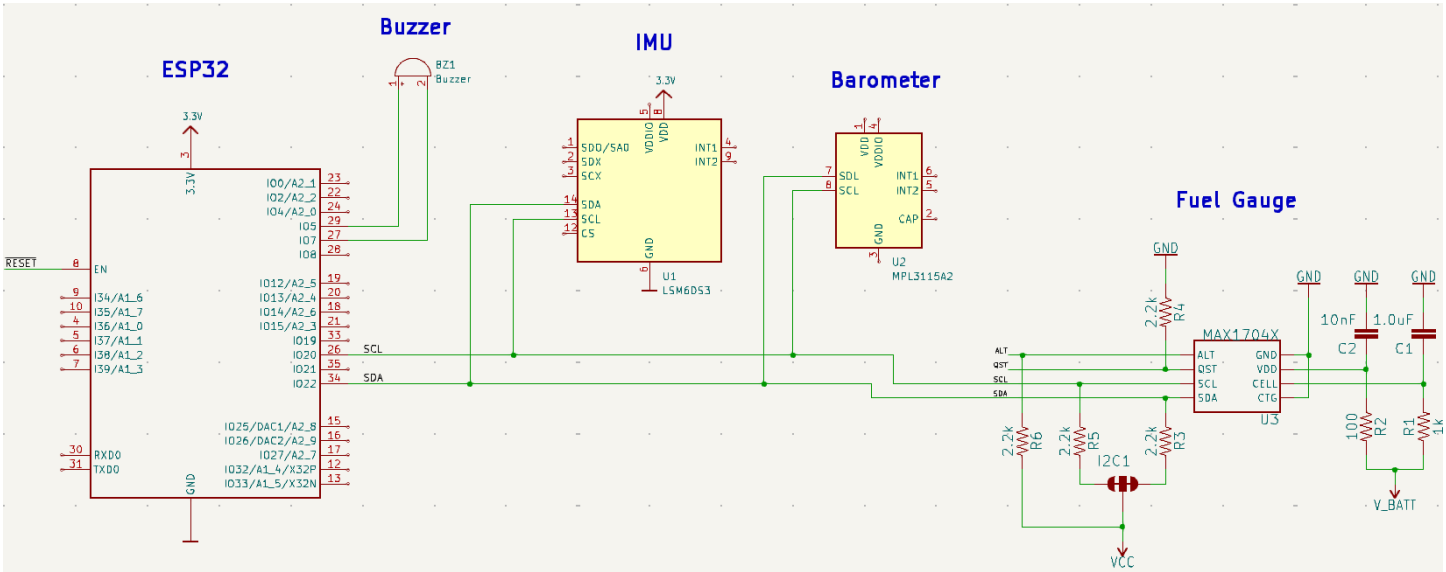

Figure 7. Microcontroller Interface Hardware Schematic

Parts

- ESP32 Feather Pico microcontroller [2]
- 6-DOF IMU sensor [3]
- Barometer [4]
- 2RGB LEDs
- 2 pushbuttons
- 1 Piezo Buzzer – PS1240 [5]
- LiPo Fuel Gauge [6]

Algorithm

Fall detection

When the device is powered on, the microcontroller continuously reads sensor data via the I2C protocol. The fall detection algorithm processes a 1.5-second window of sensor data to determine abnormal motion patterns. A fall is detected if there is a sharp acceleration in the –z direction or a rapid decrease in altitude. The algorithm also considers pre-fall and post-fall behavior, such as whether the user stood up after a minor fall, to avoid triggering unnecessary emergency alerts.

Once a fall is detected, the buzzer is activated and a timer is started, giving the user 2–10 seconds to respond. At the same time, a first alert signal is generated. If the user presses the cancel button within this time, the buzzer is disabled, sensor readings resume, and a cancellation signal is sent. If the timer expires without cancellation, a second alert signal is generated and the system enters the emergency state, which continues until a manual reset. During the emergency state, the buzzer remains on, and reading of sensor data is paused, except for battery monitoring, which continues.

Output configuration

LEDs display battery and connectivity status, with green indicating adequate battery or active network connection, and red indicating low battery or disconnection from the wireless network. LED brightness is

controlled via PWM, while the buzzer is driven using PWM with alternating square waves applied to two pins to amplify the sound, ensuring it is loud enough to alert the user in an emergency.

Theory of Operation

The microcontroller hardware interface in RESCU is responsible for reading sensor data and monitoring battery status, processing the data with an algorithm to accurately detect falls, generating relevant alerts, and controlling physical output components. The microcontroller selected for RESCU is the **ESP32 Feather Pico**, chosen primarily for its support of the **Inter-Integrated Circuit (I2C)** peripheral and its integrated **Bluetooth and Wi-Fi** capabilities for wireless communication.

RESCU uses two sensors: a 6-degree-of-freedom Inertial Measurement Unit (IMU) and a barometer. The IMU measures linear acceleration and rotational velocity, which allows the system to detect both the magnitude of a fall (through z-axis acceleration) and the potential severity of injury (through angular velocity). For example, if a person falls with minimal rotation of the device, it is less likely that the head impacted the ground, reducing the chance of a serious emergency. The barometer provides supplemental data by measuring absolute altitude via air pressure, improving the accuracy of fall detection.

The I2C interface is used for reading sensor data and battery status due to its simplicity and efficiency. I2C requires only two signal lines, SDA and SCL, to connect multiple devices, which minimizes wiring complexity and pin usage, a crucial factor in wearable device design. Although I2C operates at slower speeds than other protocols like SPI, the maximum sampling rate required by RESCU is 20kbps, far below I2C's standard-mode speed of 100 kbps and fast-mode speed of 400 kbps [7]. Therefore, standard mode is sufficient and preferred, as it consumes less power.

Specification Measurements

[Design Document 3][Every specification above should be validated with measurements]

Standards

- **IEC 62368-1:** Provides safety guidelines for battery-powered wearable electronic devices, addressing electrical, thermal, and mechanical hazards to protect users. [8]
- **I^2C communication:** Defines the communication protocol used between the IMU sensor and the ESP32 microcontroller, enabling low-power and reliable sensor data transfers through clock and data signals. [9]

[Design Document 3][Finalized]

Subsystem 3: Application Network Communication – Alexis Haas

Subsystem Diagram

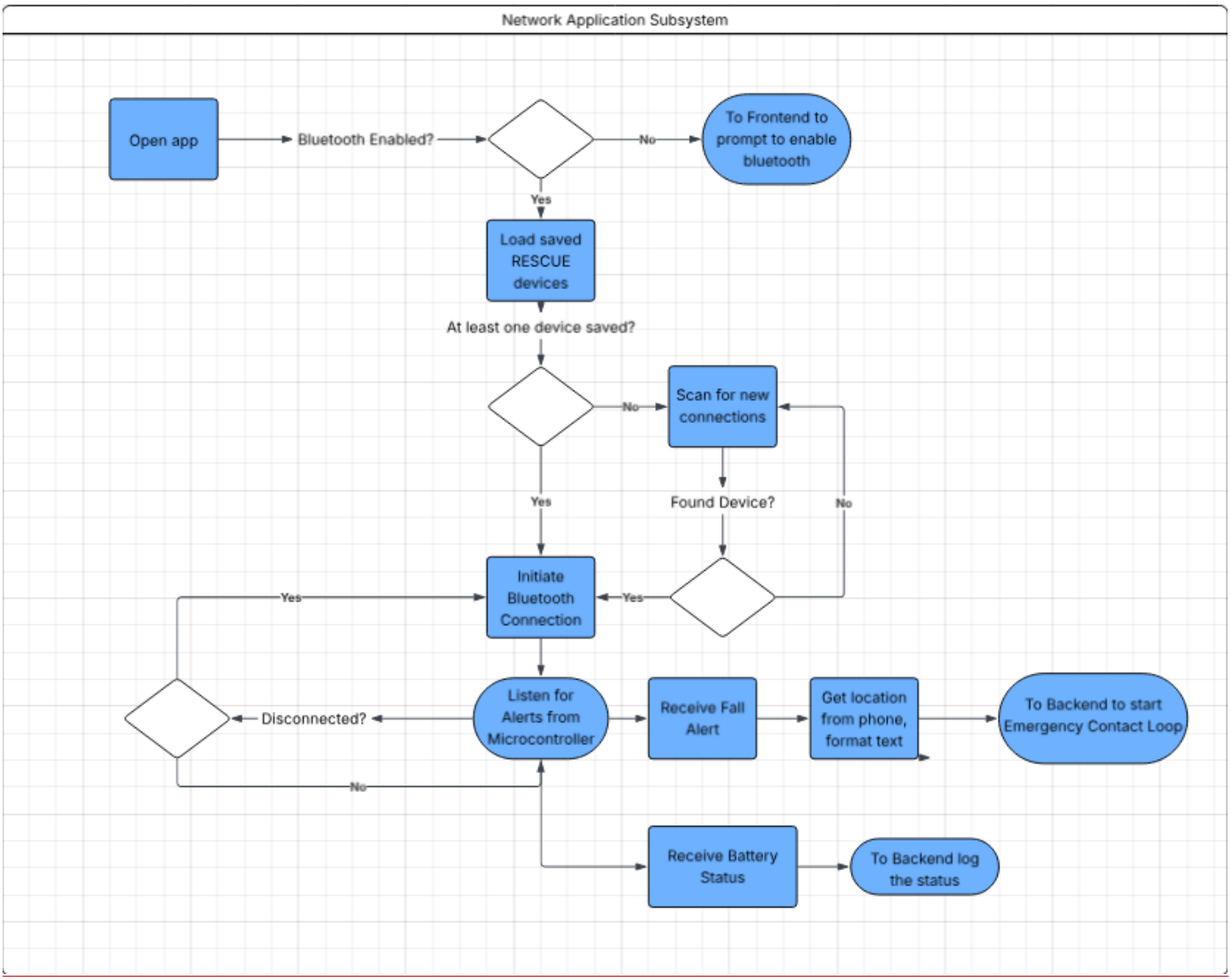

Figure 8. Application Network Communication Activity Diagram

Specifications

- **Specification 1:** The mobile application shall establish and maintain a Bluetooth Low Energy (BLE) connection with the device within 5 seconds of the device entering a 20-foot range.
    - **Requirement:** The system shall maintain reliable wireless communication between the wearable device and the paired mobile application within an expected residential operating range.
    - **Justification:** The device serves as the primary unit for fall detection, so timely and reliable communication between the device and the app is critical to system functionality. Ensuring a stable connection within this range reflects the expected usage scenario in which users typically carry or keep their mobile device nearby. Continuous connectivity supports ongoing device monitoring and reduces the risk of missed fall alerts due to communication failure, improving overall system reliability.

- **Specification 2:** Upon receiving a fall alert over BLE from the device, the mobile application shall transmit a structured alert payload to the Application Backend with 100% required data field completeness.
    - **Requirement:** Upon confirmation of a fall event, the system shall securely transmit a complete and actionable alert payload to backend to initiate emergency response procedures.
    - **Justification:** Since the Application Backend initiates the emergency contact loop, it must receive complete and correctly formatted information in order to function properly. A structured payload with full data field completeness ensures that critical information such as device identifier and timestamp are always included. Missing or malformed data could delay emergency response or prevent proper logging of fall events. Ensuring 100% completeness reduces the risk of system-level failure during emergency situations.
- **Specification 3:** The mobile application shall request the user's location using the Chrome Geolocation API with a 2-second timeout upon receiving a validated fall alert and include latitude/longitude in the backend payload when available; otherwise, the payload shall mark location as "unavailable."
    - **Requirement:** Upon confirmation of a fall event, the system shall securely transmit a complete and actionable alert payload to backend to initiate emergency response procedures.
    - **Justification:** Location data is essential for emergency contacts to assess the severity and context of a fall event. Including valid GPS coordinates in transmitted alerts enables caregivers or emergency responders to quickly determine the user's location, particularly if the event occurs outdoors or away from home.

Subsystem Interactions

This subsystem will interact with the Microprocessor Hardware Interface and the Application Back-End. When the device detects a fall event, the Microprocessor Hardware Interface subsystem transmits a fall alert to the mobile application via Bluetooth Low Energy (BLE). The Application Network Communication subsystem establishes and maintains this BLE connection, receives the alert signal, and verifies successful data transmission. After validating incoming data from the device, this subsystem securely forwards alert and device status information to the Application Backend subsystem. The backend then processes this information to initiate the emergency contact loop, store system logs, and manage user records. This subsystem does not directly manage user interface components, instead it acts as the communication bridge between the embedded hardware system and the cloud-based backend infrastructure.

Core ECE Design Tasks

- **ECE 20875 – Python for Data Science:** Basic Python commands and analysis tools
- **ECE 30861 – Software Engineering:** Software design process, software testing, and AWS services
- **ECE 30864 – Software Engineering Tools:** Application design and project management tools (JIRA)
- **ECE 36200 – Microprocessor Systems:** Embedded protocols and hardware software integration concepts
- **ECE 46300 – Intro to Computer Communication Networks:** Network protocols and transmission reliability

Schematics (Pseudo-Code)

BLE Connection Setup:

Check if Bluetooth is enabled

If Bluetooth disabled: prompt user to enable Bluetooth, then re-check

Scan for nearby RESCU devices

If no device found: continue scanning until timeout, then notify user “device not found”

If device found: connect to device over BLE

Subscribe to BLE notifications for:

Fall alert characteristic

Device status characteristic (battery / connection / health)

Set connection state = “connected”

BLE Connection Monitoring (Runs continuously while app is active):

While app is running:

If BLE connection is lost:

Set connection state = “disconnected”

Attempt reconnection

If reconnection succeeds: resubscribe to notifications and set state = “connected”

If reconnection fails after timeout: notify user “connection lost” and stop retry loop

Receive and Validate BLE Notification:

On BLE notification received:

Decode payload

Determine message type (Fall Alert vs Status Update)

If Status Update:

Validate required status fields (ex. battery %, device id, timestamp)

If fields missing: fill from cached device id and app timestamp

Forward status to backend (or cache locally if backend unavailable)

If Fall Alert:

Validate required alert fields

If fields missing or malformed:

Populate device id from stored paired-device record

Populate timestamp from application time

Mark alert as “validated” and proceed to Location and Backend handoff

Get Location and Backend Handoff:

Request user location via Chrome Geolocation API

If location permission denied OR timeout OR unavailable:

Set location = “unavailable”

If location retrieved:

Set location = (latitude, longitude)

Construct structured alert payload for backend

Send alert payload to Application Back-End via HTTPS request

If backend request fails:

Retry send (limited retries)

If still failing: queue alert for resend and notify app layer that backend transmission failed

Parts

- AWS Services [1]

Algorithm

BLE Communication: When the user opens the device interface within the mobile application, a check is done to ensure Bluetooth is enabled. Once confirmed, the app will scan nearby devices using any saved past device names as well as new devices. Once a device is detected and selected by the user, a Bluetooth Low Energy (BLE) connection is established, and the app subscribes to BLE notifications from the device. We will support 2 types of notifications, fall alerts, and device status alerts. While the connection is active, the app will continuously monitor the state of the connection to be reflected by the Connection LED and in the device interface page within the app. If the connection drops, the app returns scanning for devices and attempts reconnection until the device is successfully reconnected, or after a set timeout has expired at which point the user will get a notification informing them the connection to the device has been lost.

Receiving and Validating Alert: When a BLE notification is received from the device, the application reads and decodes the data received. The application verifies that all required fields are present and correctly formatted. If any fields are missing or malformed, the application will use backup methods to obtain the values, such as a stored device ID and application-based timestamp. Once the alert is confirmed to be valid, the application proceeds to location acquisition and backend handoff.

Get location and Backend Handoff: After a validated fall alert is received, the application requests the user's current geographic location using Chromes Geolocation API. If location access is denied, unavailable, or times out, the application proceeds with the alert with the location marked “unavailable” so that emergency processing is not blocked. If location acquisition is successful, the location field is populated with its value, and the complete formatted alert is transmitted to the AWS hosted backend subsystem to initiate the Emergency Contact Loop.

Theory of Operation

The Network Communication Subsystem is responsible for reliable data transmission between the RESCU wearable device, the application layer, and the backend server infrastructure. It manages Bluetooth Low Energy (BLE) communication with the device and secure network communication with the backend server. This subsystem ensures that alert and status data are transmitted accurately, completely, and with minimal latency. During normal operation, the device advertises a predefined BLE service. When within range, the subsystem establishes a BLE connection and subscribes to notifications for fall detection events, manual emergency triggers, and device status updates. BLE communication is event-driven, allowing the subsystem to receive structured alert packets asynchronously without continuous polling. The subsystem monitors connection status and performs automatic reconnection attempts if the BLE link is interrupted. When an alert packet is received from the device, the subsystem verifies the packet fields and confirms that required data fields are present. Valid packets are formatted into a structured network payload for backend transmission. If packet validation fails, the relevant fields will be populated from secondary sources, for example, pulling a timestamp from the application rather than receiving it from the device. Following packet validation, the subsystem transmits the alert payload to the backend server using a secure HTTPS request. Transport Layer Security (TLS) is used to ensure data confidentiality and integrity during transmission. The subsystem monitors request success and handles transmission failures through retry logic to reduce the likelihood of lost emergency alerts. Throughout system operation, the Network Communication Subsystem remains in a monitoring state, maintaining BLE connectivity with the wearable device and network readiness for backend communication. This architecture ensures timely and secure delivery of fall detection data while maintaining separation between device communication and backend processing responsibilities.

Specification Measurements

[Design Document 3][Every specification above should be validated with measurements]

Standards

- **IEEE 802.11 & Bluetooth Low Energy (BLE):** Ensures reliable and standardized wireless communication between the ESP32 microcontroller and the mobile application for transmitting fall-detection alerts. [2] [3]
- **OASIS CAP v1.2:** This is a standard for structured emergency alerts. We will structure our fall event alerts to fit this standard. [4]
- **NFPA 72 – Chapter 24:** This is a standard for the speed of emergency messaging alerts. We will use this to ensure our emergency alerts are delivered quickly. [5]

[Design Document 3][Finalized]

Subsystem 4: Application Front-End – Reed Baker

Subsystem Diagram

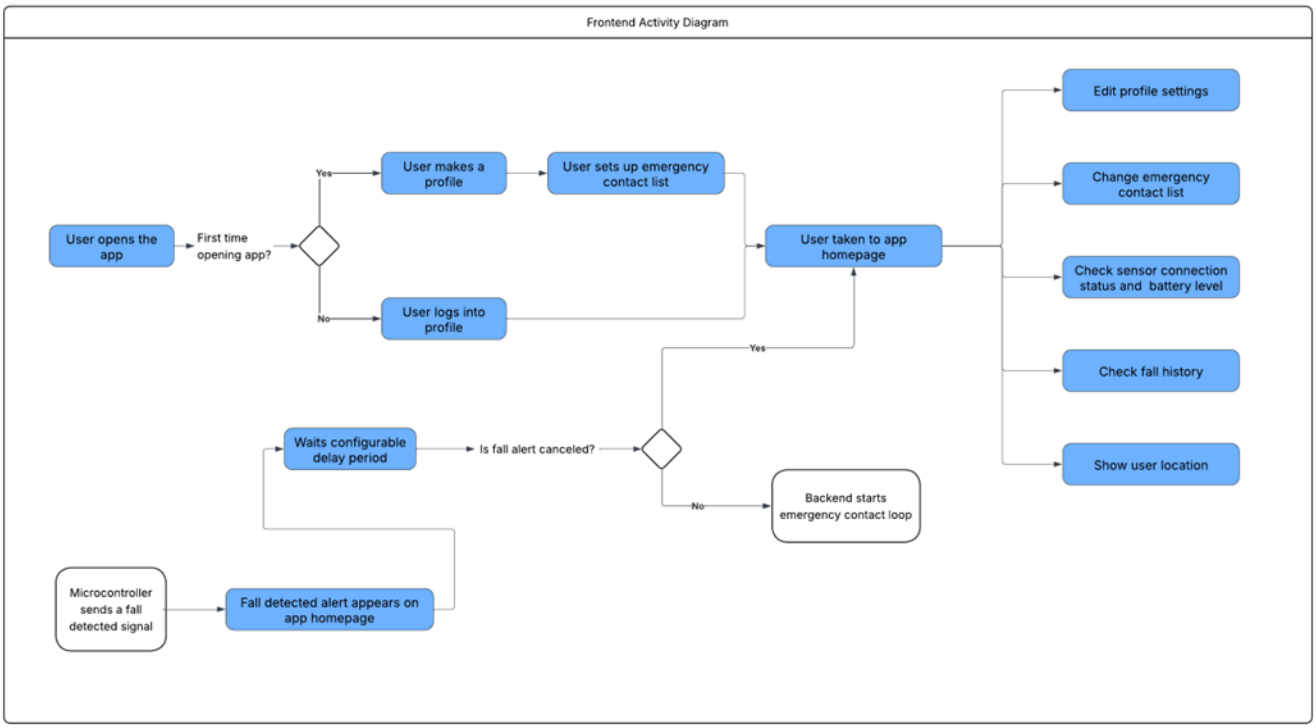

Figure 9. Application Front-End Activity Diagram

Specifications

- **Specification 1:** The mobile application shall render a visible fall alert notification within ≤ 2 seconds of receiving a valid fall alert from the device. The alert should also tell the user that to cancel the notification they need to press the SOS button on the sensor.
    - **Requirement:** Upon detection of a potential fall event, the system shall display an alert on the mobile application and wait for a configurable delay period during which the alert may be canceled.
    - **Justification:** Since this device serves as an emergency medical device, it is imperative that the emergency alert consistently goes off within a set amount of time. This is to make sure that the user receives help within a reasonable about of time, assuming they injured themselves during a fall. Additionally, in the event that the user did not injure themselves during a fall, or that the fall was a false positive, it is important that we have a way for the user to easily disengage the alarm to prevent the emergency contact loop from running.
- **Specification 2:** The mobile device shall allow the user to enter and edit Emergency Contact Profiles and configure a priority contact list and shall attempt to contact in that order when a fall event occurs.
    - **Requirement:** The system shall allow users to create, modify, and delete emergency contact information and configure alert preferences.
    - **Justification:** When editing Emergency Contact Profiles, all updates must be saved correctly and reflected immediately in the system. For example, if a contact's phone number is changed, the emergency contact loop should call the new number rather than the old one. Likewise, if the priority order of the emergency contacts is updated, the loop must follow the new order during the notification process.

- **Specification 3:** The mobile application will follow ISO/IEC 25010 standards to ensure our application is reliable, recoverable, and operable.
    - **Requirement:** The system shall have a robust and well-made mobile application that is user-friendly.
    - **Justification:** The application must meet a baseline level of operational stability and quality. This includes minimizing application crashes and ensuring consistent performance under expected operating conditions. Additionally, the UI design must be intuitive, efficient, and accessible to a diverse user base, including individuals with disabilities. The ISO/IEC 25010 framework provides a structured model that defines measurable characteristics for these traits. Aligning the mobile application with this standard ensures that attributes are clearly defined and testable.

Subsystem Interactions

The front-end subsystem interacts specifically with two other subsystems. The first one is the **Application Network Communication** subsystem which the front-end receives a fall alert from. This signal is what the front-end system uses to determine if it needs to display the fall detected emergency alert and start the count-down process to contact the backend to start the emergency contact loop. The other subsystem the front-end subsystem interacts with is the **Application Backend** subsystem. This subsystem is what the front-end queries to start the emergency contact loop when the count-down process reaches 0 and hasn't been canceled.

Core ECE Design Tasks

- **ECE 30864: Software Engineering Tools-** Using APIs and designing website interfaces
- **ECE 30862: Object Oriented Programming-** Object oriented programming
- **VIP 17911: Freshman VIP Group-** Website design and user interface design
- **VIP 47920: Senior VIP Group-** User interfacing and customer design

Schematics (Pseudo-Code)

User opens the app

Options to sign-up/login

First time user:

User selects to sign up

Sets up user profile

Sets up emergency contact list

Sends to app homepage

User already has a profile:

User selects to login

Inputs login information

Sends to app homepage

App homepage:

Able to select user profile to view/edit current information

Able to alter emergency contact list

Able to check sensor location and battery status

Able to check fall history

Able to check user location

Fall detection alert received:

Displays emergency alert message on app

Waits configurable delay period

User cancels emergency alert:

Sends to app homepage

User doesn't cancel emergency alert:

Queries backend to start the emergency contact loop

Parts

- AWS Services [1]

Theory of Operation

The front page of the mobile application will be implemented using JavaScript. One primary justification for this choice is that JavaScript is easy to integrate with Python, which is what the backend of our mobile application will be using. Another reason is that JavaScript is the most widely used programming language to make mobile applications, which means that there will be a lot of resources available to reference when designing the application. Furthermore, because of JavaScript's widespread usage, it has a large suite of features and tools available to help with every kind of project. Having such a wide array of tools available to solve any problem that occurs is invaluable when working on a project and allows us as a team much more versatility.

Specification Measurements

[Design Document 3][Every specification above should be validated with measurements]

Standards

- **WCAG 2.1 Level AA**: This is a standard for ADA-compliant webpages. We need our application interface to be accessible to all users. [2]
- **ISO/IEC 25010:** This is a standard for measuring quality and robustness of software. We need to ensure our application is reliable, recoverable, and operable. [3]

[Design Document 3][Finalized]

Subsystem 5: Application Back-End – Ishana Thakre

Subsystem Diagram

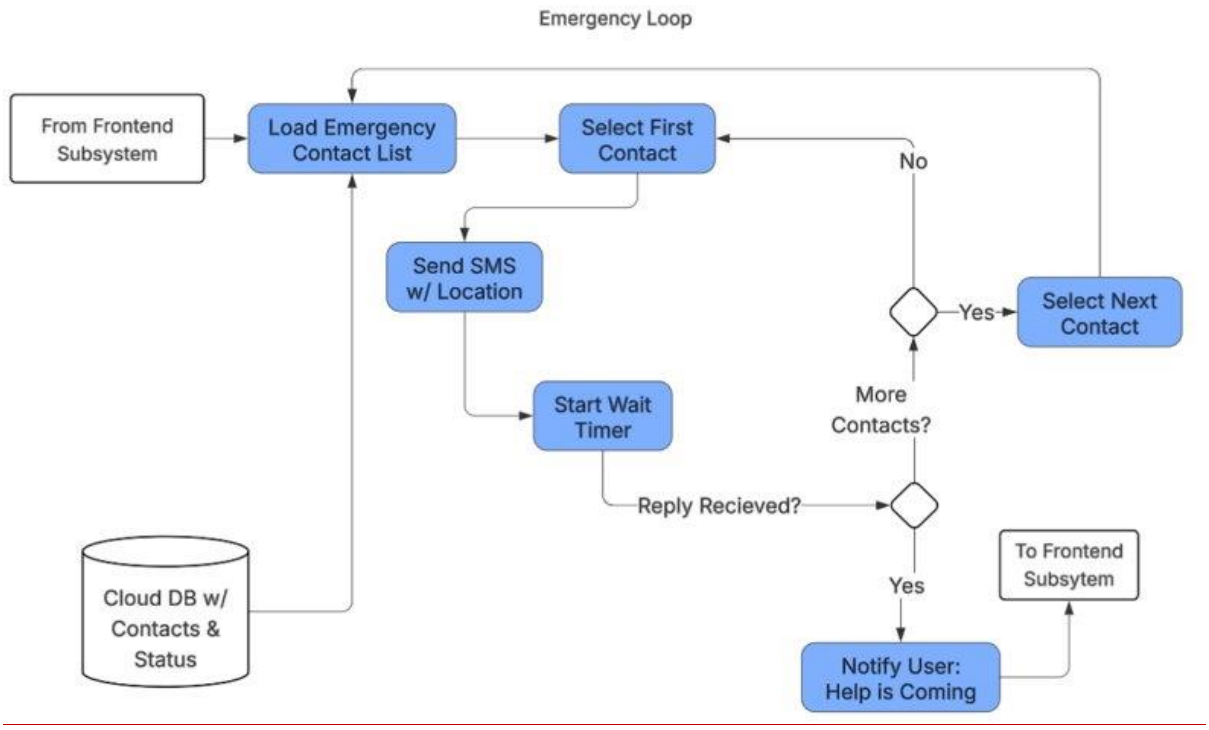

Figure 10. Application Back-End Activity Diagram

Specifications

- **Specification 1:** All stored data is encrypted using the AES-256 Advanced Encryption Standard
  - **Requirement:** The system shall protect and manage stored user and device data using secure, authenticated, and scalable infrastructure.
  - **Justification:** We are collecting sensitive Patient Health Information, so we must abide by the HIPAA data protection rules to ensure that this information is secure and never publicly accessible, even in the case of a cyberattack. AES-256 is a widely accepted encryption standard, so we believe this will be sufficient for protecting stored data.
- **Specification 2:** Data storage and retrieval should be scalable, supporting at least 10 simultaneous users, caregivers, and emergency contacts
  - **Requirement:** The system shall protect and manage stored user and device data using secure, authenticated, and scalable infrastructure.
  - **Justification:** Our application will have one primary user for each fall detection device, and a varying number of caregivers and emergency contacts for each primary user. The application must be able to function properly under the worst-case scenario in which every primary user and caregiver is trying to access the app simultaneously. For the scale that we are building at, we believe that 10 simultaneous users is sufficient, and we will ensure that this can be easily scaled up without too many changes to the infrastructure.
- **Specification 3:** Errors, crashes, and security issues within the application should be monitored and detected within 5 seconds of occurrence

    - **Requirement:** The system shall detect fall events during normal daily wearable use with high accuracy and minimal false positives.
    - **Justification:** We must ensure that there are no fatal errors in the application, as an error can cause the emergency loop system to fail. All tasks should have an appropriate fallback mechanism, and any errors should have detailed logging, so the development team can mitigate the risk as soon as possible. Any anomalies will be documented, and specific tests will be implemented to reduce the risk of this occurring.
- **Specification 4:** Emergency alerts are guaranteed to reach recipient at least once, and alert loop only stops when reply is received
    - **Requirement:** The system shall notify the configured emergency contacts in a defined priority order until an affirmative response is received.
    - **Justification:** Since these alerts will notify contacts of a possible emergency medical situation, it is critical that we ensure messages are delivered to the correct recipient in a timely manner. We will have mechanisms to check that messages are being received, and that the loop will not end even if an error occurs in a different subsystem. Due to the emergency nature of these alerts, if we cannot confirm that a message has been delivered, we would rather have the message delivered twice than never.

Subsystem Interactions

This subsystem interacts with the Application Front-End and Application Network Communication subsystems. When a user interacts with the frontend of the application (ex. Clicking a button), the signal will be sent to the backend and handled in this subsystem. This subsystem will also use alert data from the microprocessor that is formatted by the application network communication subsystem.

Core ECE Design Tasks

- **ECE 30861 – Software Engineering:** Software design process, software testing, and AWS services.
- **ECE 20875 – Python for Data Science:** Basic Python commands and analysis tools.
- **ECE 30864 – Software Engineering Tools:** Application design and project management tools (JIRA)
- **ECE 40400 – Introduction to Computer Security:** Essential cybersecurity practices and common threats.

Schematics (Pseudo-Code)

Application Initialization (When new user attempts to create account):

Receive entered user data from frontend

If Primary User data:

Create unique key for user and save information to database

If Caregiver data:

Find associated Primary User in database

Notify Primary User about Caregiver addition

If confirmed, link Primary User information and save Caregiver information to database

Emergency Loop (Occurs when fall is confirmed, and user does not manually cancel alert)

Receive fall alert with GPS coordinates

Load emergency contact list from database

While affirmative reply not received:

Send SMS message to next emergency contact

Wait 60 seconds for affirmative reply

If no further contacts: Begin again with first emergency contact

Notify user – help is coming

User Interaction (Occurs only when user requests data from frontend)

Receive request from frontend

Request necessary data from network subsystem (battery status, connection status, etc.)

Store data in database (overwrite previous stored value if needed)

Send data to frontend

Parts

- AWS Services [1]

Algorithm

Database Storage and Communication: The backend has multiple NoSQL databases containing Primary User, Caregiver, Emergency Contact, and Device information. All information is linked to a unique key assigned to each Primary User in the database, and all information will be queried through this key to ensure that information is never made available to unauthorized users. All database communications use the official AWS SDK for Python (boto3). There are also rigorous and well-tested authentication algorithms to ensure that malicious users cannot access the application or its data. When any user creates an account, they are required to create a strong password that follows set requirements (Capital letter, number, symbol, etc.). Passwords will never be stored in plaintext, instead they will be stored as a hash value and any entered password will be compared against the hash.

Emergency Communication Loop: Once an emergency has been detected and confirmed, the list of emergency contacts tied to the Primary User is pulled from the database, and a SMS message is sent to the contacts using AWS SNS. To avoid race conditions and possibly multiple calls to emergency services by the emergency contacts, only one contact will be notified at a time, with a 60 second timeout before the next contact is texted. If a contact replies with an affirmative response, the emergency loop will end, as we can assume that contact is getting help. The primary user is then notified that help is on the way. If the loop has run through the entire list of contacts with no response, it will begin again from the first contact. Since these are SMS messages being delivered directly to the emergency contacts' default messaging app, we can assume that after receiving repeated text messages, at least one contact will check their messages and reply.

Theory of Operation

This application is hosted through AWS and is deployed using AWS Lambda, a serverless compute service that does not require the provisioning or management of servers, operating systems, or the network layer. All data is stored in and pulled from AWS DynamoDB, a NoSQL database that allows for scalability and configurable access control. To improve searchability, we have multiple tables, all connected to a primary key linked to the device user. These two services will work together to facilitate all deployment and storage needs. The emergency communication loop will mainly use AWS SNS, a pub/sub messaging service that enables application-to-application and application-to-person communication. This allows us to send messages to verified phone numbers of emergency contacts without having to deal with complex network-layer algorithms. Finally, we will use

AWS API Gateway to create testable links for each function. This is also how the application backend will communicate with the frontend. The backend subsystem functions as a go-between for the application frontend and application network communication subsystems, as all traffic must go through the backend to ensure correct formatting and security.

For both the development and production environments of the application, we use AWS CloudWatch for real-time monitoring and anomaly detection. Within the codebase, we implement detailed error detection and logging, and CloudWatch uses this information to show developers exactly when and where an error occurred. Many errors may not be immediately detectable during user testing, so we must rely on CloudWatch to ensure that there are no hidden bugs that will cause the program, especially the emergency loop, to fail in the future.

Specification Measurements

[Design Document 3][Every specification above should be validated with measurements]

Standards

- **MQTT QoS 1:** This is a standard for messaging services, to ensure that alerts are received at least once. Since we are building a medical device, we would rather have messages delivered twice than never. [2]
- **HIPAA:** This is a law that protects sensitive patient health information. We are storing contact information and patient health information in a database. [3]
- **IEC 62304:** This is a standard that defines life cycle requirements for medical device software. Our project can be classified as a Class C device, with serious injury or death occurring if the device fails. [4]
- **NIST 800-63B:** This is a standard for authentication assurance levels and password requirements. Our project will need to have secure user registration and login methods. [5]
- **OpenAPI Specification (OAS) 3.1:** This is a standard for REST APIs. Our frontend and backend of the application will communicate through REST APIs. [6]

[Design Document 3][Finalized]

PCB Design

PCB Schematics

[Design Document 3]

PCB Layout

[Design Document 3]

Final Status

[Design Document 3]

Team Structure

Team Member 1

Reed Baker

***Contact*:** baker785@purdue.edu
Team Role: Team Lead
Major: Computer Engineering
Bio: I am from Bloomington, IN, and I am interested in software and AI/ML. In this project I will contribute by making sure that the sensor communicates with the app and database, and by contributing to the front-end design of the app.

Team Member 2

Alexis Haas

Contact: haas52@purdue.edu
Team Role: Facilitator
Major: Computer Engineering
Bio: I am from Milwaukee, WI, and I am interested in software and embedded systems. I will be contributing to this project by implementing the Bluetooth communication interface between the wearable device and the mobile application, as well as ensuring reliable data formatting for the alert and handoff to the backend system.

Team Member 3

Ishana Thakre

Contact: ithakre@purdue.edu
Team Role: Purchasing
Major: Computer Engineering
Bio: I am from Bentonville, AR, and I am interested in software engineering and application development. I will be working with the cloud services used to deploy our app, and handle backend functions.

Team Member 4

Haejune Kwon

Contact: kwon196@urdue.edu
Team Role: Internal Communication
Major: Computer Engineering
Bio: I am from South Korea, and I am interested in digital design and embedded system software. With such interest and experience, I will design the microcontroller interface to control the hardware components and communicate with the mobile application.

Team Member 5

Jacob Hilton

Contact: hilton19@purdue.edu
Team Role: External Communication
Major: Electrical Engineering
Bio: I am from Plainfield, IN, and I am interested in signal processing and device calibration. I have had experience in both of these fields in my internships, so I look forward to being able to apply these skills towards the hardware design for this project.

Bibliography

Executive Description

[1] Centers for Disease Control and Prevention. (n.d.). *Older adult falls data*. https://www.cdc.gov/falls/data-research/index.html

User Story

[1] Centers for Disease Control and Prevention. (n.d.). *Disability impacts all of us* [Infographic]. https://www.cdc.gov/disability-and-health/articles-documents/disability-impacts-all-of-us-infographic.html

[2] Life Alert. (n.d.). *Life Alert official website*. https://www.lifealert.com/

[3] SeniorLiving.org. (n.d.). *Life Alert reviews*. https://www.seniorliving.org/medical-alert-systems/life-alert/reviews/

[4] FallCall Solutions. (n.d.). *FallCall Detect*. https://www.fallcall.com/Apps/FallCall-Detect

[5] SeizAlarm. (n.d.). *SeizAlarm: Seizure detection app*. https://seizalarm.com/

Subsystems

Subsystem 1 (Hardware)

[1] Adafruit. (n.d.). *Adafruit ESP32-S3 Feather with STEMMA QT / Qwiic - 8MB Flash / 2MB PSRAM* (Product ID: 5400). https://www.adafruit.com/product/5400

[2] Adafruit. (n.d.). *Adafruit ISM330DHCX + LIS3MDL FeatherWing - High Precision 9-DoF IMU* (Product ID: 4503). https://www.adafruit.com/product/4503

[3] Adafruit. (n.d.). *Adafruit BMP280 I2C or SPI Barometric Pressure & Altitude Sensor* (Product ID: 1893). https://www.adafruit.com/product/1893

[4] Adafruit. (n.d.). *Breadboard-friendly PCB mount mini speaker - 8 Ohm 0.2W* (Product ID: 160). https://www.adafruit.com/product/160

[5] Adafruit. (n.d.). *Lithium Ion Polymer Battery - 3.7v 2500mAh* (Product ID: 328). https://www.adafruit.com/product/328

[6] Adafruit. (n.d.). *USB LiIon/LiPoly charger - v1.2* (Product ID: 259). https://www.adafruit.com/product/259

[7] https://www.sparkfun.com/sparkfun-lipo-fuel-gauge.html

[8] https://www.pololu.com/product/2122/specs

[9] Bel Fuse Inc. (n.d.). *IEC 62368-1: An introduction to the new safety standard for ICT and AV equipment* [Technical paper]. https://www.belfuse.com/resource-library/tech-paper/iec-62368-1-an-introduction-to-the-new-safety-standard-for-ict-and-av-equipment

[10] NXP Semiconductors. (2021). *UM10204: I2C-bus specification and user manual*. https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Subsystem 2 (Microcontroller)

[1] Robinovitch, S. N., Normandin, S. C., Stotz, P., & Maurer, J. D. (2005). *Time requirement for young and elderly women to move into a position for breaking a fall with outstretched hands*. *The Journals of Gerontology, Series A*, 60(12), 1553–1557. https://doi.org/10.1093/gerona/60.12.1553

[2] Adafruit. (n.d.). *Adafruit ESP32-S3 Feather* (Product ID: 5400). https://www.adafruit.com/product/5400

[3] Adafruit. (n.d.). *Adafruit ISM330DHCX + LIS3MDL FeatherWing* (Product ID: 4503). https://www.adafruit.com/product/4503

[4] Adafruit. (n.d.). *Adafruit BMP280 Barometric Pressure Sensor* (Product ID: 1893). https://www.adafruit.com/product/1893

[5] Adafruit. (n.d.). *Mini speaker - 8 Ohm 0.2W* (Product ID: 160). https://www.adafruit.com/product/160

[6] SparkFun Electronics. (n.d.). *SparkFun LiPo Fuel Gauge* [Product page]. SparkFun. https://www.sparkfun.com/sparkfun-lipo-fuel-gauge.html

[7] Espressif Systems. (n.d.). *I2C — ESP32 API reference* (ESP-IDF Programming Guide). https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html

[8] Bel Fuse Inc. (n.d.). *IEC 62368-1 Safety Standard*. https://www.belfuse.com/resource-library/tech-paper/iec-62368-1-an-introduction-to-the-new-safety-standard-for-ict-and-av-equipment

[9] NXP Semiconductors. (2021). *I2C-bus specification (UM10204)*. https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Subsystem 3 (Front-end)

[1] Amazon Web Services. (n.d.). *AWS products and services*. https://aws.amazon.com/products/

[2] World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

[3] International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). https://www.iso.org/standard/78176.html

Subsystem 4 (Back-end)

[1] Amazon Web Services. (n.d.). *AWS products and services*. https://aws.amazon.com/products/

[2] OASIS. (2014). MQTT version 3.1.1. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[3] U.S. Department of Health & Human Services. (n.d.). *HIPAA security rule*. https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html

[4] International Electrotechnical Commission. (2006). *Medical device software — Software life cycle processes* (IEC 62304:2006). https://www.iso.org/obp/ui/#iso:std:iec:62304:ed-1:v1:en

[5] National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). https://pages.nist.gov/800-63-3/sp800-63b.html

[6] OpenAPI Initiative. (2021). *OpenAPI specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0.html

Subsystem 5 (Network)

[1] Amazon Web Services. (n.d.). *AWS products and services*. https://aws.amazon.com/products/

[2] IEEE 802.11 Working Group. (n.d.). *IEEE 802.11 wireless LAN technology*. https://www.ieee802.org/11/

[3] Bluetooth Special Interest Group. (2022). *The Bluetooth LE primer (v1.2.0)*. https://www.bluetooth.com/wp-content/uploads/2022/05/the-bluetooth-le-primer-v1.2.0.pdf

[4] OASIS. (2010). *Common Alerting Protocol version 1.2*. https://docs.oasis-open.org/emergency/cap/v1.2/CAP-v1.2-os.html

[5] National Fire Protection Association. (2016). *NFPA 72: Chapter 24 — Emergency communication systems*. https://ifmacentralohio.org/images/meeting/101316/nfpa_chapter_24__emergency_communication_systems.pdf

Appendices

[This section is mainly designed for code. You can directly generate a somewhat decent display of your code file or psuedo code by using the template provided below. You can have as many appendix as you want. In the document, you can refer to the code posted here instead of pasting the whole code in the body.]

Appendix A: [NAME]

[Insert code]